Defensa TP Integrador DAO — Grupo 100

Nov 1, 2026 · @Darián Rodriguez Dieguez

Resumen

La defensa es el **viernes 6/11/2026**: hay que correr Huellitas en vivo, mostrar dónde está cada patrón en el código y justificar por qué se usó ahí y por qué no en otros lugares. Las preguntas pueden ir a cualquiera de las tres, así que el reparto define quién lleva cada bloque, no quién sabe cada tema.

Contenido: reparto de bloques (sección 2), chequeo técnico (sección 3), guion de la demo (sección 4), dónde está cada cosa en el código (sección 5) y preguntas probables con respuesta (sección 6).

Reparto

Propuesta para unos 15 minutos; ajustenla entre ustedes. Cada bloque junta una parte de la demo con el código y los patrones que la sostienen, así quien muestra la pantalla también explica el porqué.

Bloque	Quién	Tiempo	Qué cubre
1. Presentación y análisis	Florencia	2 min	Problema, casos de uso, modelo de dominio, arquitectura en capas
2. Atención clínica	Florencia	3 min	Demo pasos 1 a 4: clientes, mascotas, turnos, vacunas, derivación. Validaciones de turnos y vacunas
3. Cadena de custodia	Darián	5 min	Demo pasos 5 a 9: empaquetado, despacho, control, recepción con rechazo. Patrones State y Observer de la muestra y del remito, bitácoras y triggers
4. Laboratorio, facturación y persistencia	Antonela	3 min	Demo pasos 10 y 11: resultado, comprobante, trazabilidad. Singleton con transacciones, Repository, esquema SQL

Bloque	Quién	Tiempo	Qué cubre
5. Reportes y pruebas	Antonela	2 min	Demo pasos 12 y 13: Template Method y los 6 reportes, CSV, los 42 tests

Preguntas: responde quien llevó ese bloque; si no sabe, la que sí sabe completa sin pisarla.

Preparación técnica

La demo depende de datos generados con la fecha del día, así que la base se regenera la mañana de la defensa.

Antes del viernes (cada una en su compu)

- [ ] `git pull` en la carpeta del repo y `py -m pip install -r requirements.txt`
- [ ] `py main.py --datos-prueba` : abre http://127.0.0.1:5000 y la app carga
- [ ] `py -m unittest -v` : termina con `Ran 42 tests ... OK`
- [ ] Correr la demo completa (sección 4) al menos una vez sola
- [ ] Definir qué compu se usa para exponer y cuál es el plan B (otra compu con todo andando)

El viernes, antes de entrar

- [ ] `py main.py --datos-prueba` esa misma mañana: deja muestras sin empaquetar, un remito Emitido y otro Anulado listos para mostrar
- [ ] VS Code abierto en el proyecto con estas pestañas: `dominio/muestra.py` , `dominio/remito.py` , `servicios/observadores.py` , `reportes/base.py` , `persistencia/conexion.py` , `persistencia/repositorios.py` , `persistencia/esquema.sql` , `servicios/logistica.py`
- [ ] Zoom del navegador y de VS Code al 125 %, notificaciones apagadas, otras pestañas cerradas
- [ ] El documento explicativo abierto en `docs/` por si piden los diagramas

La app no necesita internet: Bootstrap y Chart.js están incluidos.

Guion de la demo

Dos recorridos: primero el camino feliz de una muestra nueva (de la atención al comprobante) y después una falla de cadena de frío sobre el remito Emitido que dejan los datos de prueba. Todo esto ya lo probé contra datos recién generados.

Florencia · atención clínica

1. **Inicio**: indicadores (atenciones, muestras en circuito, tasa de rechazo, facturación). Decir: “el núcleo es la trazabilidad de la muestra”.
2. **Mascotas**: intentar cargar una mascota sin responsable → error. Abrir la historia clínica de otra: atenciones, vacunas, análisis y esquema de vacunación.
3. **Agenda**: reservar un turno a un veterinario en un horario que ya tiene ocupado → error de superposición.
4. **Atención clínica**: elegir una mascota que muestre la alerta de vacunas (por ejemplo Rocco) y un veterinario de Clínica Centro. Registrar con **Registrar y derivar a laboratorio**, elegir Hemograma con prioridad Urgente → se crea la muestra Extraída.

Darián · cadena de custodia

5. **Empaquetado** (Clínica Centro): elegir la muestra nueva y primero la conservadora de 15 a 25 °C → “no es apta”. Después la de 2 a 8 °C → remito Emitido + precinto. Muestra → Empaquetada.
6. **Detalle del remito**: despachar (transportista cualquiera, 5 °C) → remito y muestra En tránsito. Mostrar la bitácora de estados del remito.
7. **Recepción**: precinto Intacto, 5 °C → “llegaron viables”; remito Recibido. Abrir `dominio/remito.py` y mostrar `Emitido.despachar()` y `EnTransito.recibir()`.
8. **Falla**: Remitos y envíos → pestaña **Por despachar** → el remito de la demo. Despachar a 5 °C, registrar un punto de control a **12 °C**, recibir con precinto Intacto a 6 °C → todas las muestras rechazadas por ruptura de cadena de frío y remito Rechazado.
9. **Bloqueo por estado**: en Resultados, intentar cargar el resultado de una de esas muestras → “No puede procesarse”. Abrir `dominio/muestra.py` y mostrar `Rechazada.analizar()`. Si preguntan por anulación: pestaña **Anulados**, el remito de la demo con su bitácora.

Antonela · laboratorio, facturación, reportes

10. **Resultados**: cargar el resultado de la muestra del paso 7 → Analizada; en **Avisos** aparece el aviso al cliente (Observer).
11. **Comprobantes**: emitir el de la atención del paso 4 → consulta + análisis. Trazabilidad: buscar el código de la muestra y recorrer su historial completo.
12. **Reportes**: Muestras rechazadas (causa y tasa por transportista) y Tiempos de tránsito; exportar a CSV.
13. **Pruebas**: en la terminal `py -m unittest -v` → 42 OK.

Si algo falla en vivo: leer el mensaje en voz alta (son errores de negocio, no caídas) y seguir con el paso siguiente.

Mapa del código

Dónde abrir cada cosa cuando digan “mostranos dónde está”. Rutas dentro de `veterinaria/`; en VS Code, `Ctrl+P` abre el archivo y `Ctrl+G` salta a la línea.

Qué	Dónde	Qué decir
State · muestra	`dominio/muestra.py`: `EstadoMuestra` l.30, subclases l.55–115, `Muestra` l.125, `_transicionar` l.177	Cada estado es una clase que define solo lo que admite; lo demás lanza `TransicionInvalidaError` (_no_permitido, l.34). `Muestra` delega en `estado_actual`
State · remito	`dominio/remito.py`: `EstadoRemito` l.30, `Emitido` l.52, `EnTransito` l.62, `Remito` l.95	Mismo esquema. `en_ruta` dice quién admite controles y recepción; solo `Emitido` tiene `anular()`
Observer · sujeto	`muestra.py` l.143 y l.146; `remito.py` l.116 y l.119	`agregar_observador` y `_notificar`: la entidad no sabe quién escucha
Observer · observadores	`servicios/observadores.py`: `RegistroTrazabilidad` l.9, `RegistroTrazabilidadRemito` l.26, `AvisoAlCliente` l.43	Bitácora con control cronológico y aviso al cliente. Se enganchan en los servicios, por ejemplo `logistica.py` l.32
Template Method	`reportes/base.py`: `Reporte` l.39, `generar()` l.50; los 6 reportes en `reportes/reportes.py`	`generar()` fija los pasos; solo `obtener_datos` es abstracto, el resto son hooks
Singleton	`persistencia/conexion.py`: `__new__` l.20, `transaccion()` l.51	Una sola conexión; transacción anidable: solo la más externa hace COMMIT o ROLLBACK
Repository	`persistencia/repositorios.py`: `Repositorio` l.18, `_repo()` l.75	Repositorio genérico por dataclass; los servicios no escriben SQL de ABM
Inmutabilidad en la base	`persistencia/esquema.sql`: triggers l.190–238	6 triggers: no se puede editar ni borrar ninguna bitácora ni cambiar un estado final
Recepción y rechazo	`servicios/logistica.py`: `recibir()` l.190, `_motivo_rechazo()` l.254	Orden: precinto → temperaturas de toda la cadena → horas de viabilidad

Qué	Dónde	Qué decir
Empaquetado	`servicios/logistica.py`: `empaquetar()` l.54	Valida conservadora, capacidad, repetidas, viabilidad; todo dentro de una transacción
Turnos sin superposición	`servicios/turnos.py`: `_superpuesto()` l.17	`inicio < fin_otro AND fin > inicio_otro`, por veterinario y por mascota
Esquema de vacunas	`servicios/vacunas.py`: `validar_aplicacion()` l.50	Especie, edad mínima, dosis única, refuerzo prematuro
Fechas coherentes	`dominio/entidades.py`: `fecha_no_futura()` l.409	Ningún hecho de la trazabilidad puede fecharse en el futuro
Errores en la interfaz	`web/util.py`: `validar()` l.38	Decorador: el error de negocio se muestra como alerta y el formulario conserva lo cargado
Pruebas	`tests/test_negocio.py` (34) y `tests/test_web.py` (8)	`py -m unittest -v`

Preguntas probables

La cátedra anticipó que preguntan qué patrones usamos, para qué, dónde los usaríamos y por qué no en otro lugar. Las respuestas están en primera persona del grupo para practicarlas en voz alta.

Patrones

Pregunta	Respuesta corta
¿Qué patrones usaron?	Cinco: State, Observer, Template Method, Singleton y Repository. La consigna pedía dos; State y Observer sostienen la trazabilidad, que es lo que evalúan.
¿Por qué State en la muestra?	Porque lo que se puede hacer depende del estado: una muestra Rechazada no se analiza, una Extraída no se recibe. Sin State tendríamos `if estado == ...` repetidos en cada servicio; con State la regla vive en un solo lugar, `Rechazada.analizar()`.

Pregunta	Respuesta corta
¿Y en el remito?	Lo mismo: solo un remito Emitido se despacha o se anula, y solo uno En tránsito admite controles y recepción. Primero lo hicimos con una tabla de transiciones y lo pasamos a State por consistencia y porque sí cambia de comportamiento.
¿Por qué no State en turnos o conservadoras?	Tienen tres estados sin comportamiento propio, solo una etiqueta. State conviene cuando el comportamiento cambia con el estado; acá serían tres clases vacías.
¿Para qué Observer?	Cada cambio de estado tiene que quedar en la bitácora y, si la muestra se analiza o rechaza, avisar al cliente. La muestra solo notifica; no sabe que existen la base ni los avisos. Agregar un mail real es sumar un observador sin tocar `Muestra`.
¿Qué resuelve Template Method?	Los 6 reportes siguen los mismos pasos: validar fechas, obtener datos, procesar, resumir y graficar. `generar()` fija el orden y cada reporte redefine solo lo suyo. La pantalla los trata a todos igual.
¿Por qué Singleton?	Para que una transacción abarque varios servicios: el empaquetado toca remito, bitácora, precinto, muestras y conservadora, y si algo falla se deshace todo. Con una conexión por servicio no se podría.
¿Desventaja del Singleton?	Es estado global: los tests lo reinician con `BaseDatos.reiniciar()` y el servidor corre con un solo hilo. Para una app local de un usuario es aceptable.
¿Para qué Repository?	Separa el SQL de altas y modificaciones de la lógica de negocio. Es uno genérico que mapea cada dataclass con su tabla. Las consultas con joins de listados y reportes quedan en servicios y reportes.

Pregunta	Respuesta corta
¿Dónde usarían Strategy?	En las reglas de rechazo de la recepción o en precios con descuentos. Hoy son tres reglas fijas en un método y el precio es un dato en `config.py`, así que no lo justificaba. Con reglas por tipo de análisis usaríamos Chain of Responsibility.
¿Usaron Iterator o Factory?	Iterator lo trae Python (listas, `for`, generadores). Factory está implícito: `_repo()` crea las clases de repositorio y los diccionarios `ESTADOS` registran los estados.

Base de datos y reglas

Pregunta	Respuesta corta
¿Cómo garantizan la trazabilidad inmutable?	En dos niveles: el código solo inserta en las bitácoras y la base tiene triggers que abortan cualquier UPDATE o DELETE sobre ellas y cualquier cambio de un estado final.
¿Qué pasa si el precinto llega roto?	Se rechazan todas las muestras del remito, el remito queda Rechazado y `Rechazada.analizar()` impide procesarlas. Se avisa al cliente para una nueva extracción.
¿Qué más se controla al recibir?	En orden: precinto, temperaturas de toda la cadena (salida, controles y llegada) contra el rango del análisis, y horas desde la extracción contra su viabilidad.
¿Cómo evitan turnos superpuestos?	Dos intervalos se pisan si `inicio_a < fin_b` y `inicio_b < fin_a`; se controla por veterinario y por mascota, ignorando los cancelados.
¿Cómo controlan las vacunas?	Cada tipo de vacuna tiene especie, edad mínima y refuerzo. Al registrar la atención se valida especie, edad y refuerzo prematuro, y se avisa lo pendiente o vencido.
¿Por qué SQLite y sin ORM?	La consigna pide base de datos; SQLite no requiere servidor y el SQL propio muestra el modelo relacional y los triggers.

Pregunta	Respuesta corta
¿Cómo probaron que funciona?	42 pruebas automáticas: 34 de reglas de negocio y 8 que recorren la web, incluido el circuito completo desde los formularios.